

Practical 1

```
graph = {
    'A' : ['B','C'],
    'B' : ['D', 'E'],
    'C' : ['F'],
    'D' : [],
    'E' : ['F'],
    'F' : []
}
visited = [] # List to keep track of visited nodes.
queue = [] #Initialize a queue
def bfs(visited, graph, node):
    visited.append(node)
    queue.append(node)
    while queue:
        s = queue.pop(0)
        print (s, end = " ")
        for neighbour in graph[s]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

# Driver Code
print("Following is the Path using Breadth-First Search")
bfs(visited, graph, 'A')

graph = {
    'A' : ['B','C'],
    'B' : ['D', 'E'],
    'C' : ['F'],
    'D' : [],
    'E' : ['F'],
    'F' : []
}
visited = set() # Set to keep track of visited nodes of graph.
def dfs(visited, graph, node): #function for dfs
    if node not in visited:
        print (node)
        visited.add(node)
        for neighbour in graph[node]:
            dfs(visited, graph, neighbour)

# Driver Code
print("Following is the Path using Depth-First Search")
dfs(visited, graph, 'A')
```

Output

```
student@student-OptiPlex-3070: ~  
student@student-OptiPlex-3070:~$ python3 AI1.py  
Following is the Path using Breadth-First Search  
A B C D E F Following is the Path using Depth-First Search  
A  
B  
D  
E  
F  
C  
student@student-OptiPlex-3070:~$
```

Practical 2

```
class Graph:
    def __init__(self, adjacency_list):
        self.adjacency_list = adjacency_list
    def get_neighbors(self, v):
        return self.adjacency_list.get(v, [])
    def h(self, n):
        H = {
            'A': 1,
            'B': 1,
            'C': 1,
            'D': 1
        }
        return H[n]

# A* Algorithm
def a_star_algorithm(self, start_node, stop_node):
    open_list = set([start_node])
    closed_list = set([])

    g = {start_node: 0}

    # Parents dictionary
    parents = {start_node: start_node}

    while open_list:
        n = None
        # Find node with lowest f = g + h
        for v in open_list:
            if n is None or g[v] + self.h(v) < g[n] + self.h(n):
                n = v

        if n is None:
            print("Path does not exist!")
            return None

        if n == stop_node:
            path = []
            while parents[n] != n:
                path.append(n)
                n = parents[n]
            path.append(start_node)
            path.reverse()
            print("Path found:", path)
            return path

        for (m, weight) in self.get_neighbors(n):
            if m not in open_list and m not in closed_list:
```

```

        open_list.add(m)
        parents[m] = n
        g[m] = g[n] + weight
    else:
        if g.get(m, float('inf')) > g[n] + weight:
            g[m] = g[n] + weight
            parents[m] = n

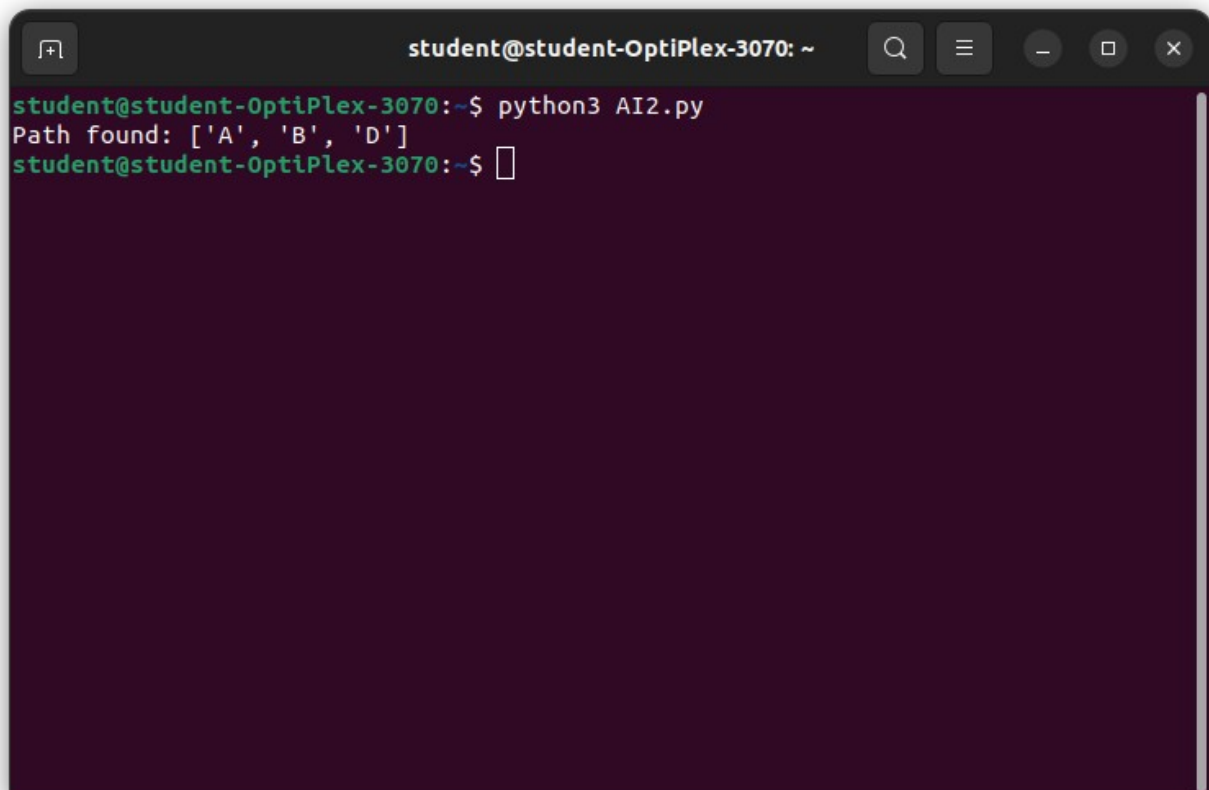
        if m in closed_list:
            closed_list.remove(m)
            open_list.add(m)

    # Move node from open to closed
    open_list.remove(n)
    closed_list.add(n)
    print("Path does not exist!")
    return None

adjacency_list = {
    'A': [('B', 1), ('C', 3), ('D', 7)],
    'B': [('D', 5)],
    'C': [('D', 12)],
    'D': []
}
graph1 = Graph(adjacency_list)
graph1.a_star_algorithm('A', 'D')

```

Output



```

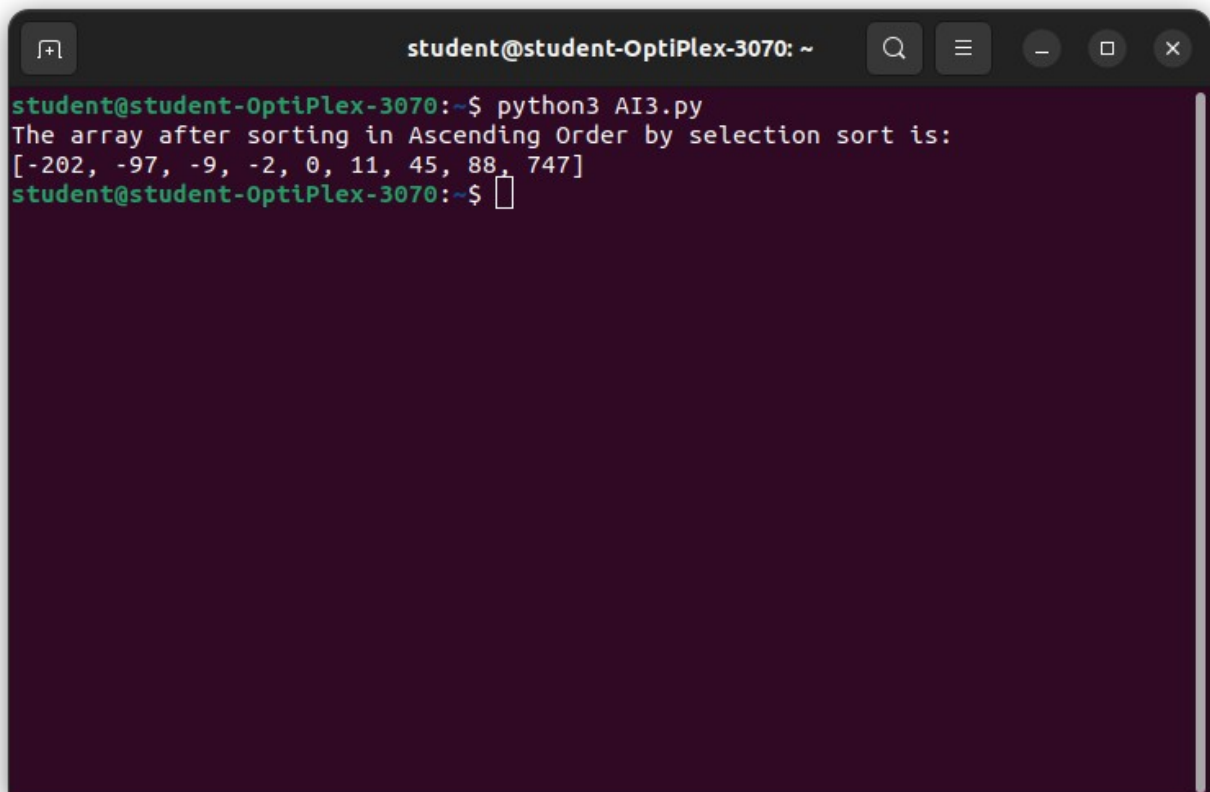
student@student-OptiPlex-3070: ~
student@student-OptiPlex-3070:~$ python3 AI2.py
Path found: ['A', 'B', 'D']
student@student-OptiPlex-3070:~$ 

```

Practical 3

```
def selectionSort(array, size):  
    for ind in range(size):  
        min_index = ind  
  
        for j in range(ind + 1, size):  
            # select the minimum element in every iteration  
            if array[j] < array[min_index]:  
                min_index = j  
  
        # swapping the elements to sort the array  
        array[ind], array[min_index] = array[min_index], array[ind]  
  
# Main program  
arr = [-2, 45, 0, 11, -9, 88, -97, -202, 747]  
size = len(arr)  
  
selectionSort(arr, size)  
  
print("The array after sorting in Ascending Order by selection sort is:")  
print(arr)
```

Output



A terminal window titled "student@student-OptiPlex-3070: ~" with standard window controls. The terminal shows the execution of a Python script named "AI3.py". The output of the script is displayed in a light blue font on a dark background. The output consists of two lines: a message indicating the array after sorting and the sorted array itself.

```
student@student-OptiPlex-3070:~$ python3 AI3.py  
The array after sorting in Ascending Order by selection sort is:  
[-202, -97, -9, -2, 0, 11, 45, 88, 747]  
student@student-OptiPlex-3070:~$
```

Practical 4

Python program to solve N Queen Problem using backtracking

N = 4

```
def printSolution(board):
    for i in range(N):
        for j in range(N):
            print(board[i][j], end=' ')
        print()

# A utility function to check if a queen can be placed
def isSafe(board, row, col):
    # Check this row on left side
    for i in range(col):
        if board[row][i] == 1:
            return False

    # Check upper diagonal on left side
    for i, j in zip(range(row, -1, -1), range(col, -1, -1)):
        if board[i][j] == 1:
            return False

    # Check lower diagonal on left side
    for i, j in zip(range(row, N, 1), range(col, -1, -1)):
        if board[i][j] == 1:
            return False

    return True

def solveNQUtil(board, col):
    # Base case: If all queens are placed
    if col >= N:
        return True

    # Try placing queen in all rows one by one
    for i in range(N):
        if isSafe(board, i, col):
            # Place queen
            board[i][col] = 1

            # Recur to place rest
            if solveNQUtil(board, col + 1):
                return True

            # Backtrack
            board[i][col] = 0

    return False
```

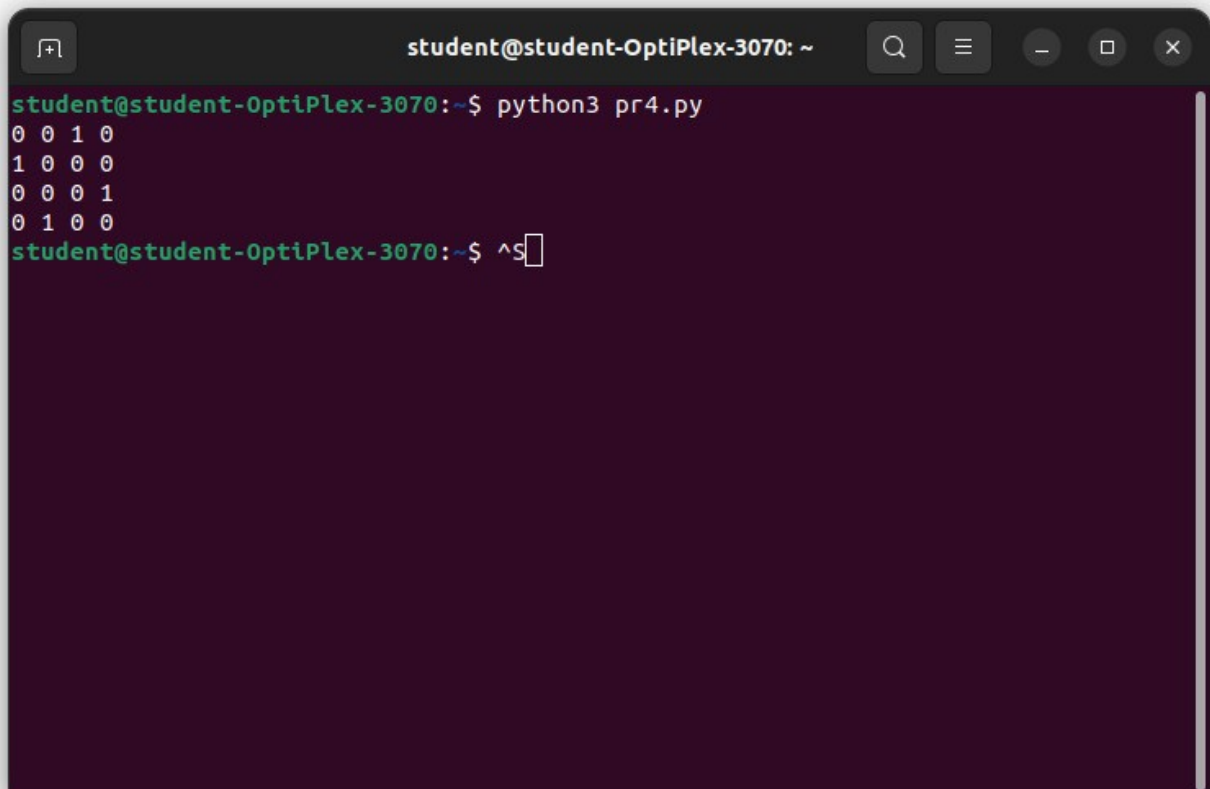
```
def solveNQ():
    board = [
        [0, 0, 0, 0],
        [0, 0, 0, 0],
        [0, 0, 0, 0],
        [0, 0, 0, 0]
    ]

    if not solveNQUtil(board, 0):
        print("Solution does not exist")
        return False

    printSolution(board)
    return True

# Driver code
solveNQ()
```

Output



```
student@student-OptiPlex-3070: ~
student@student-OptiPlex-3070:~$ python3 pr4.py
0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0
student@student-OptiPlex-3070:~$ ^S
```

Practical 5

Simple Chatbot Program

```
def greet(bot_name, birth_year):
    print("Hello! My name is {0}.".format(bot_name))
    print("I was created in {0}.".format(birth_year))

def remind_name():
    print('Please, remind me your name.')
    name = input()
    print("What a great name you have, {0}!".format(name))

def guess_age():
    print('Let me guess your age.')
    print('Enter remainders of dividing your age by 3, 5 and 7.')

    rem3 = int(input())
    rem5 = int(input())
    rem7 = int(input())

    age = (rem3 * 70 + rem5 * 21 + rem7 * 15) % 105

    print("Your age is {0}; that's a good time to start programming!".format(age))

def count():
    print('Now I will prove to you that I can count to any number you want.')

    num = int(input())
    counter = 0

    while counter <= num:
        print("{0} !".format(counter))
        counter += 1

def test():
    print("Let's test your programming knowledge.")
    print("Why do we use methods?")
    print("1. To repeat a statement multiple times.")
    print("2. To decompose a program into several small subroutines.")
    print("3. To determine the execution time of a program.")
    print("4. To interrupt the execution of a program.")

    answer = 2
    guess = int(input())

    while guess != answer:
        print("Please, try again.")
```



```

guess = int(input())

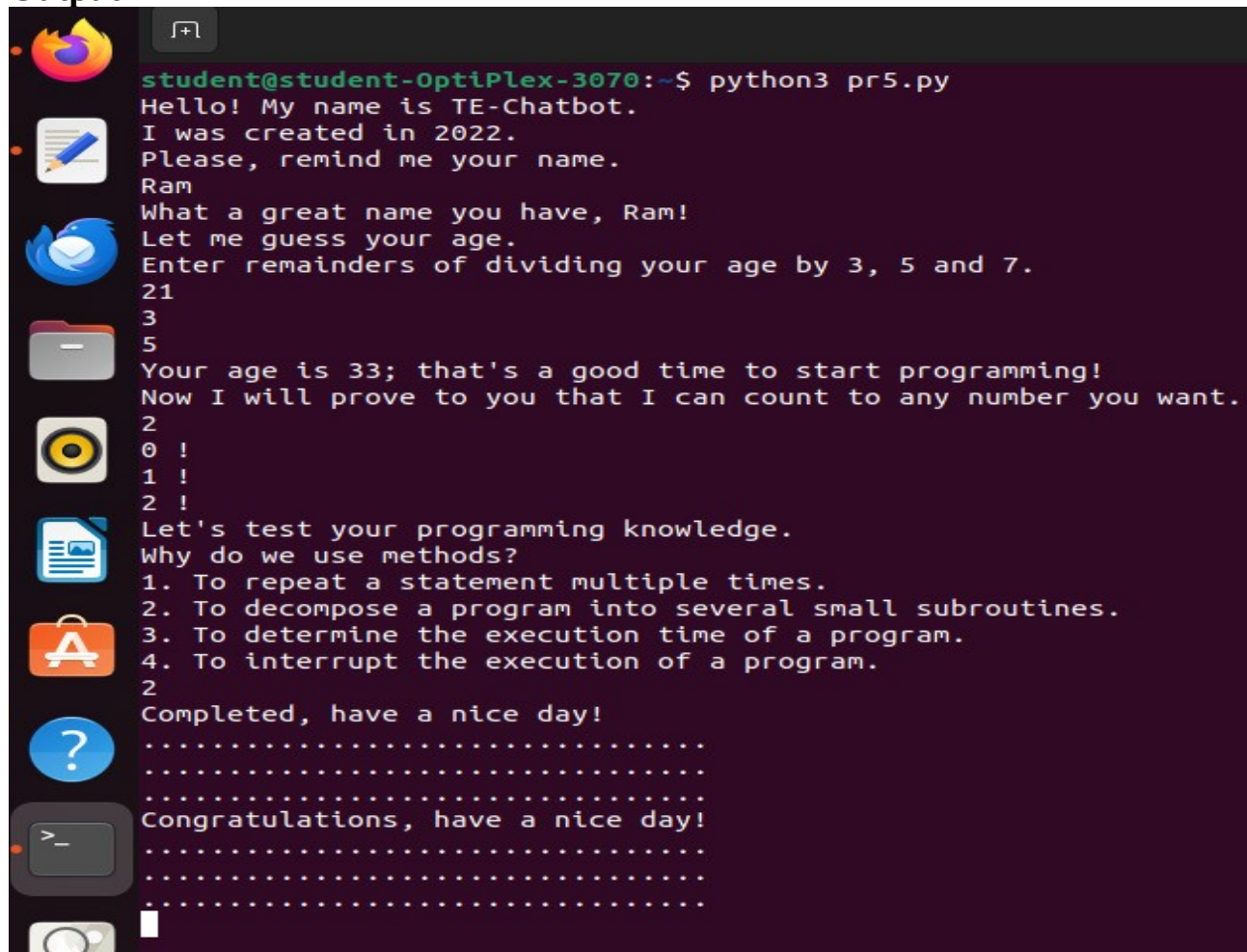
print('Completed, have a nice day!')
print('.....')
print('.....')
print('.....')

def end():
    print('Congratulations, have a nice day!')
    print('.....')
    print('.....')
    print('.....')
    input()

# Main program execution
greet('TE-Chatbot', '2022') # change it as needed
remind_name()
guess_age()
count()
test()
end()

```

Output



```

student@student-OptiPlex-3070:~$ python3 pr5.py
Hello! My name is TE-Chatbot.
I was created in 2022.
Please, remind me your name.
Ram
What a great name you have, Ram!
Let me guess your age.
Enter remainders of dividing your age by 3, 5 and 7.
21
3
5
Your age is 33; that's a good time to start programming!
Now I will prove to you that I can count to any number you want.
2
0 !
1 !
2 !
Let's test your programming knowledge.
Why do we use methods?
1. To repeat a statement multiple times.
2. To decompose a program into several small subroutines.
3. To determine the execution time of a program.
4. To interrupt the execution of a program.
2
Completed, have a nice day!
.....
.....
.....
Congratulations, have a nice day!
.....
.....
.....

```

Practical 6

Help Desk Management System

Define a dictionary of common problems and their solutions

```
problem_dict = {  
    "Printer not working": "Check that it's turned on and connected to the network",  
    "Can't log in": "Make sure you're using the correct username and password",  
    "Software not installing": "Check that your computer meets the system requirements",  
    "Internet connection not working": "Restart your modem or router",  
    "Email not sending": "Check that you're using the correct email server settings"  
}
```

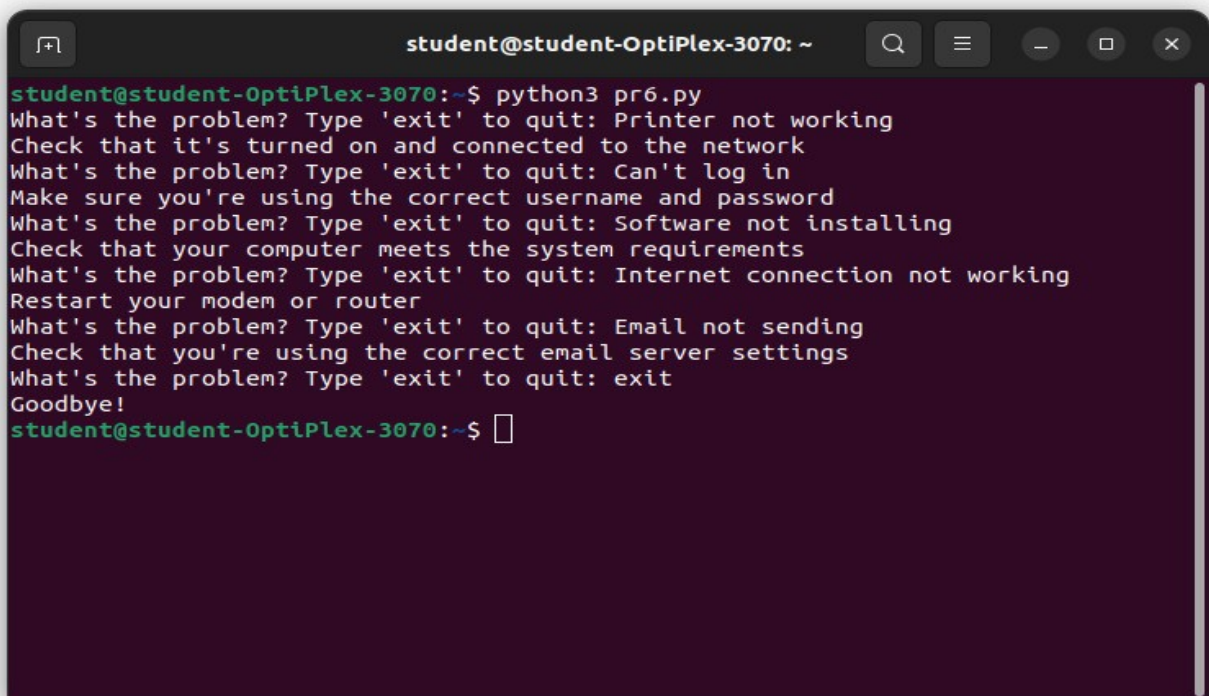
Define a function to handle user requests

```
def handle_request(user_input):  
    if user_input.lower() == "exit":  
        return "Goodbye!"  
    elif user_input in problem_dict:  
        return problem_dict[user_input]  
    else:  
        return "I'm sorry, I don't know how to help with that problem."
```

Main loop to prompt user for input

```
while True:  
    user_input = input("What's the problem? Type 'exit' to quit: ")  
    response = handle_request(user_input)  
    print(response)  
  
    if user_input.lower() == "exit":  
        break
```

Output



```
student@student-OptiPlex-3070: ~  
student@student-OptiPlex-3070:~$ python3 pr6.py  
What's the problem? Type 'exit' to quit: Printer not working  
Check that it's turned on and connected to the network  
What's the problem? Type 'exit' to quit: Can't log in  
Make sure you're using the correct username and password  
What's the problem? Type 'exit' to quit: Software not installing  
Check that your computer meets the system requirements  
What's the problem? Type 'exit' to quit: Internet connection not working  
Restart your modem or router  
What's the problem? Type 'exit' to quit: Email not sending  
Check that you're using the correct email server settings  
What's the problem? Type 'exit' to quit: exit  
Goodbye!  
student@student-OptiPlex-3070:~$
```

